Simulacro de Examen DI — Unidad 2 (WPF)

TechFix: Gestor de Reparaciones · Práctica intensiva (XAML parcial + Lógica completa en C#)

⏱️ **Tiempo objetivo: 60 min**

📁 **Proyecto: Documents/github/DI/SimulacroExamenDI**

🎯 **Foco: Ventanas, ListView, Botones, Validación**

> **Qué ya tienes hecho:** La estructura base del proyecto (`MainWindow.xaml` y `VentanaDetalle.xaml`) está creada sin nada de código C#. Tu trabajo es completar los pequeños `TODO XAML` marcados en la interfaz y programar toda la lógica en `.cs` .

Parte 1: Modelo de datos y remates en XAML

~8 min

1. Crea la clase `Reparacion.cs` con las propiedades automáticas: `Cliente` (`string`), `Dispositivo` (`string`), `FechaEntrada` (`DateTime`), `Urgencia` (`int`), `EsGarantia` (`bool`), `Estado` (`string` , por defecto `"Pendiente"`) y `NotasTecnicas` (`string` , por defecto `""`).
2. En `MainWindow.xaml` , resuelve los `TODO XAML` estructurales:
   - Añade el `<ComboBoxItem Content="Consola"/>` dentro de `cbDispositivo` .
   - Añade la columna **Estado** en `lvPendientes` enlazada a `{Binding Estado}` .
   - Completa `lvFinalizadas` (en la segunda pestaña) con un `GridView` o `ItemTemplate` que muestre Cliente, Dispositivo y Estado.
3. En `VentanaDetalle.xaml` , enlaza `Text="{Binding Dispositivo}"` en `tbDispositivo` .
4. *(A medida que avances)* Enlaza en ambos XAML los eventos `Click` , `Checked` , `ValueChanged` y `TextChanged` marcados con `TODO XAML` .

Parte 2: Eventos de controles y protección inicial

~7 min

1. **Slider + ProgressBar + Label:** En el evento `ValueChanged` de `slUrgencia` , sincroniza `pbUrgencia.Value` y muestra en `lblUrgencia` el texto `"Nivel: X / 5"` .
2. **Protección obligatoria:** Evita el típico `NullReferenceException` durante `InitializeComponent()` comprobando si `pbUrgencia == null || lblUrgencia == null` .
3. **RadioButtons:** Al marcar `rbGarantia` , haz visible `lblAvisoGarantia` (`Visibility.Visible`); al marcar `rbNormal` , ocúltalo (`Visibility.Collapsed`).

Parte 3: Validación de formulario y Alta en ListView

~15 min

En `MainWindow.xaml.cs` , declara dos colecciones observables: `listaPendientes` y `listaFinalizadas` , y enlázalas en el constructor a `lvPendientes.ItemsSource` y `lvFinalizadas.ItemsSource` .

Al pulsar `btnRegistrar` , valida **antes de crear el objeto** (si algo falla: `MessageBox.Show(...)` y `return;`):

1. `txtCliente` no puede estar vacío ni ser solo espacios (`string.IsNullOrWhiteSpace`).
2. Debe haber un dispositivo elegido en `cbDispositivo` . *(Ojo: guarda solo el texto limpio, no `"System.Windows.Controls.ComboBoxItem: ..."`)*.
3. Debe haber una fecha en `dpFecha` y **no puede ser mayor que hoy** (`DateTime.Today`).
4. Debe estar marcado `rbNormal` o `rbGarantia` .

Si todo es válido: crea el objeto `Reparacion` , añádelo a `listaPendientes` , actualiza `lblTotalPendientes` y limpia todos los campos del formulario (`SelectedIndex = -1` , `SelectedDate = null` , `IsChecked = false` , `slUrgencia.Value = 1`).

Parte 4: ListView — Filtrar, Ordenar y Eliminar ~15 min

1. **Filtrar (`txtBuscar`):** En `TextChanged` , obtén la vista con `CollectionViewSource.GetDefaultView(lvPendientes.ItemsSource)` , asigna `vista.Filter` para mostrar solo los clientes que contengan el texto buscado (ignorando mayúsculas/minúsculas) y llama a `vista.Refresh()` .
2. **Ordenar por columna:** En `GridViewColumnHeader_Click` , lee `col.Tag.ToString()` y ordena usando `vista.SortDescriptions` (limpiando las anteriores antes de añadir la nueva).
3. **Eliminar (`btnEliminar`):** Comprueba que haya una fila seleccionada en `lvPendientes` (`SelectedItem is Reparacion sel`). Si no, avisa. Si hay selección, pide confirmación con `MessageBoxButton.YesNo` ; solo elimina y actualiza `lblTotalPendientes` si pulsa `Yes` .

Parte 5: Ventana secundaria y mover entre ListViews ~15 min

1. **Ver / Editar (`btnVerEditar`):**
   - Si hay elemento seleccionado en `lvPendientes` , abre `VentanaDetalle` en modo **modal** (`ShowDialog()`) pasándole la `Reparacion` por constructor.
   - En `VentanaDetalle` : asigna `this.DataContext = _reparacion;` y precarga `txtNotas.Text` y `cbEstado.Text` .
   - En `btnGuardar` : si el estado elegido es `"Esperando pieza"` , valida que `txtNotas` no esté vacío. Guarda los cambios en el objeto, establece `this.DialogResult = true;` y cierra.
   - En `btnCancelar` : establece `this.DialogResult = false;` .
   - De vuelta en `MainWindow` : si `ShowDialog() == true` , llama a `lvPendientes.Items.Refresh();` para que se actualice la columna *Estado* en pantalla.
2. **Finalizar (`btnFinalizar`):** Con confirmación `YesNo` , cambia `Estado = "Finalizada"` , quita la reparación de `listaPendientes` , añádela a `listaFinalizadas` y actualiza ambos contadores.
3. **Reabrir (`btnReabrir` en pestaña 2):** Mueve la reparación seleccionada en `lvFinalizadas` de vuelta a `listaPendientes` cambiando su `Estado = "Pendiente"` y actualizando ambos contadores.

Resumen rápido de controles (x:Name)

Ventana	Controles disponibles
MainWindow (Pestaña 1)	`tcPrincipal` , `txtCliente` , `cbDispositivo` , `dpFecha` , `rbNormal` , `rbGarantia` , `slUrgencia` , `pbUrgencia` , `lblUrgencia` , `lblAvisoGarantia` , `btnRegistrar` , `txtBuscar` , `lvPendientes` , `lblTotalPendientes` , `btnVerEditar` , `btnFinalizar` , `btnEliminar`
MainWindow (Pestaña 2)	`lvFinalizadas` , `lblTotalFinalizadas` , `btnReabrir`
VentanaDetalle	`tbCliente` , `tbDispositivo` , `cbEstado` , `txtNotas` , `btnGuardar` , `btnCancelar`

> **Checklist de examen:** Si terminas este ejercicio sin bloquearte en *(1) protección de InitializeComponent*, *(2) validaciones con return*, *(3) Filter/SortDescriptions*, *(4) ShowDialog + DialogResult + Items.Refresh()* y *(5) mover entre ObservableCollections*, estás preparado para el examen.